Fundamentos de Programación II

Descripción Ver entrega

Límite de entrega: martes, 1 de octubre de 2024, 10:30
Ficheros requeridos: Pruebas.java, CheckoutQueue.java, Node.java, Client.java, ContainerBase.java (Descargar)
Tipo de trabajo: Individual

Gestión de Cola en un Supermercado

Se pide completar el desarrollo de la clase `CheckoutQueue` en Java a partir del código suministrado. Cada objeto de dicha clase representa la **cola** de clientes de una caja en un supermercado.

Se quiere que la cola esté soportada por una lista encadenada. Para ello, la clase `CheckoutQueue` deriva de la clase `ContainerBase` (ya desarrollada), la cual nos suministra los atributos `first` y `last` (para apuntar, respectivamente, al primero y al último nodo de la lista), así como el método `toString()` para facilitar la visualización de la lista (el símbolo ➡️ representa el `first`, y el símbolo ⬅️ el `last`).

Se proporciona asimismo la clase `Client` (ya desarrollada), cada uno de cuyos objetos representará un cliente en cola, dotado de dos atributos públicos: `name` para el nombre y `totalCost` para el total a pagar por su compra; ambos inicializados en el constructor de la clase (recibidos en ese orden).

La lista que representa la cola está formada por nodos de la clase `Node` (ya desarrollada), que tienen como atributos: `value`, que apunta al cliente, y `next`, que apunta al siguiente nodo. El constructor de la clase `Node` tiene un único parámetro, de la clase `Cliente`.

Para completar la clase `CheckoutQueue` deberás desarrollar los siguientes métodos (pudiendo para ello añadir los atributos o métodos que fueran necesarios):

✅ Métodos a desarrollar:

1. **Método** `enqueue`
   Representa la llegada de un cliente (recibido por parámetro) a la cola de la caja. La inserción ha de ser por el final.

2. **Método** `dequeue`
   Representa que el cliente en el principio de la cola ha pasado por caja. Para ello, quita dicho cliente de la cola y lo devuelve. Además, se ha de actualizar apropiadamente lo ingresado en esa caja desde que se abrió su cola. Devuelve `null` si no había cliente en la cola.

3. **Método** `getDailySales`
   Este método nos devuelve lo ingresado en esa caja desde que se abrió.

4. **Método** `split`
   Este método está pensado para que los clientes de la cola pasen a ser atendidos en dos cajas. Para ello crea una nueva cola de caja y la devuelve tras repartir los clientes, de forma que:

   - La **cola original** conservará los clientes de posiciones **pares** (0, 2, 4, ...),
   - La **nueva cola** contendrá los clientes de posiciones **impares** (1, 3, 5, ...),
   - Manteniendo en ambas el orden relativo original de sus clientes.

   En el desarrollo de este método **se permite usar cualesquiera clases o métodos** disponibles en Java.

Notas y recomendaciones:

- **El informe de evaluación** le **proporciona el código empleado** en cada caso y el error encontrado: error de compilación 🐞, fallo en los resultados ❌, tiempo límite superado ⏰, etc.
- **Localice los errores** en su código modificando y ejecutando el fichero `Pruebas.java`. Note que **puede copiar y pegar código desde el resultado de la evaluación**. El contenido de este fichero no se evalúa.
- El tipo de prueba de evaluación empleado en este ejercicio puede requerir más de 20 segundos.
- **Despliegue** la pestaña **Comentarios** aun cuando se muestre algún mensaje en la de **Compilación** ya que en comentarios aparecerán el resultado de las pruebas realizadas.

🔄 Ejemplo de uso:

```
CheckoutQueue cola = new CheckoutQueue();
System.out.println(cola);             // ➡️null⬅️
cola.enqueue(new Client("Ana", 35));
cola.enqueue(new Client("Luis", 50));
cola.enqueue(new Client("Marta", 25));
System.out.println(cola);             // ➡️[Ana: 35]->[Luis: 50]->[Marta: 25]⬅️->null
System.out.println(cola.dequeue());   // Ana: 35
System.out.println(cola.dequeue());   // Luis: 50
System.out.println(cola.getDailySales()); // 85
System.out.println(cola);             // ➡️[Marta: 25]⬅️->None
```

Nota: las flechas ➡️ y ⬅️ señalan los nodos referenciados por los atributos `first` y `last`, mientras que `->` representa a `next`.

🛒 Queue Management in a Supermarket

You are asked to complete the development of the class **CheckoutQueue** in Java. Each object of this class represents the queue of customers at a checkout in a supermarket. The queue should be backed by a linked list.

The class **CheckoutQueue** is provided for you to complete. **CheckoutQueue** extends the already developed class **ContainerBase**, which supplies us with the attributes **first** and **last** that point to the first and last node of the list representing the queue; it also provides a **toString()** method that helps us visualize the queue. The symbol ➡️ points to the **first** of the queue, the symbol ⬅️ points to the **last**.

The class **Client** (already developed) is provided, whose objects will represent each customer in the queue. Each **Client** object has two public attributes: **name** for the customer's name and **totalCost** for the total amount to be paid for their purchase. These attributes are initialized in the class constructor, in that order.

The list representing the queue is made up of nodes of class **Node** (already developed), which have as attributes **value**, pointing to the client, and **next**, pointing to the next node. The **Node** constructor takes a single parameter: a **Client** object.

To complete the **CheckoutQueue** class you must implement the following methods (you may add any attributes or helper methods as needed):

✅ Methods to develop:

- **enqueue**
  Represents the arrival of a customer (received as a parameter) to the queue, adding them at the end.

- **dequeue**
  Represents the customer at the front of the queue having checked out. It should remove that customer and return them. Additionally, you must appropriately update the total amount collected at this checkout since the queue was opened. Returns **null** if there was no customer in the queue.

- **getDailySales**
  This method returns the total amount collected at this checkout since it was opened.

- **split**
  This method is intended to have the customers in the queue be served at two checkouts. It creates a new checkout queue and returns it after distributing the customers so that:

  - The original queue keeps the customers in even positions (0, 2, 4, …).

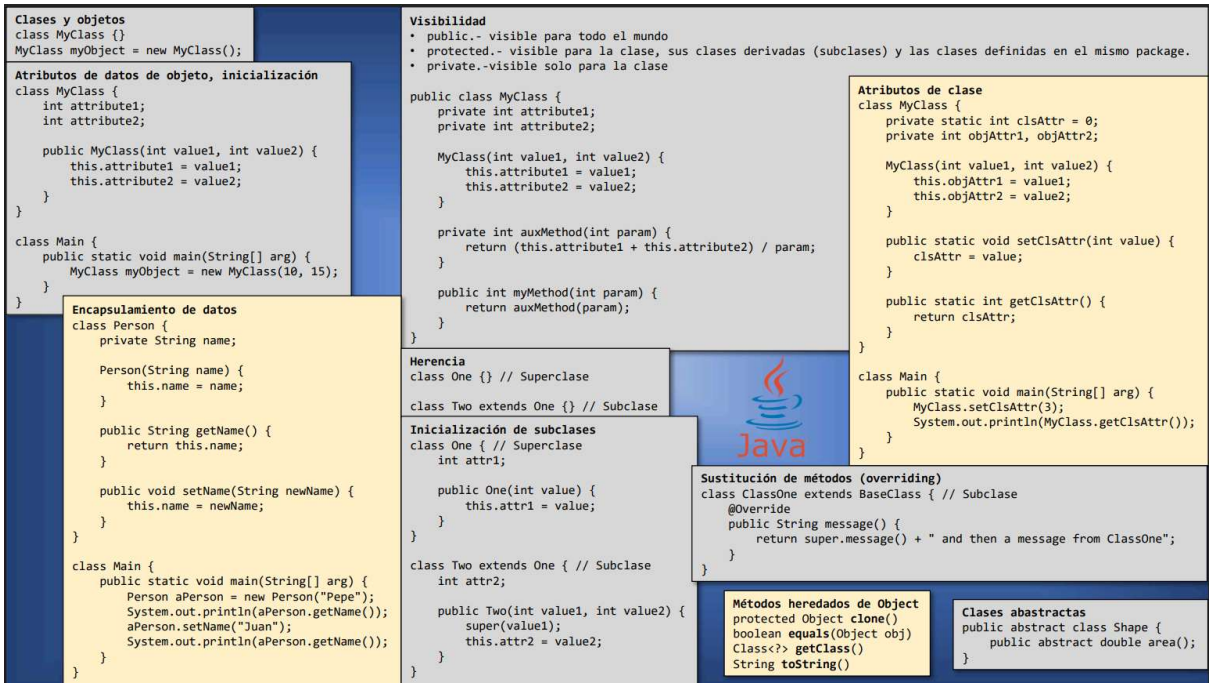
  - The new queue contains the customers in odd positions (1, 3, 5, …).

  - Both queues preserve the original relative order of their customers.
    In implementing this method you may use any classes or methods available in Java.

🧠 Notes and recommendations:

- The evaluation report provides the code used in each case and the error found: compilation error 🐞, wrong results ❌, time limit exceeded ⏰, etc.
- Locate the errors in your code by modifying and running the file **Pruebas.java**. Note that you can copy and paste code from the evaluation results. The contents of this file are not evaluated.

- The type of evaluation test used in this exercise may take more than 20 seconds.
- Open the **Comments** tab even if a message appears in the **Compilation** tab, since the test results will appear in **Comments**.



Ficheros requeridos

Pruebas.java

```java
public class Pruebas {
    public static void main (String[] args) {
        CheckoutQueue cola = new CheckoutQueue();
        System.out.println(cola);             // ➡️null⬅️
        cola.enqueue(new Client("Ana", 35));
        cola.enqueue(new Client("Luis", 50));
        cola.enqueue(new Client("Marta", 25));
        System.out.println(cola);             // ➡️[Ana: 35]->[Luis: 50]->[Marta: 25]⬅️->null
        System.out.println(cola.dequeue());   // Ana: 35
        System.out.println(cola.dequeue());   // Luis: 50
        System.out.println(cola.getDailySales()); // 85
        System.out.println(cola);
    }
}
```

CheckoutQueue.java

```java
// Desarrolle aquí su código

```

Node.java

```java
public class Node{
    public Client value;
    public Node next;
    public Node(Client value){
        this.value = value;
    }
}

```

Client.java

```java
public class Client{
    public final String name;
    public final int totalCost;
    public Client(String name, int totalCost) {
        this.name = name;
        this.totalCost = totalCost;
    }
}

```

ContainerBase.java

```java
public class ContainerBase{
    protected Node first;
    protected Node last;
    public String toString() {
        // No se muestra
    }
}

```